PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

UE24CS341A - SOFTWARE ENGINEERING

JACKFRUIT MINI-PROJECT

TRAFFIC MANAGEMENT SYSTEM (SIMULATION)

Detailed Project Plan, Work Division and Development Roadmap

Team Member	SRN	Primary Functional Ownership
Aarya	PES1UG24CS496	Vehicle and Traffic Input Management
Vijeth	PES1UG24CS531	Adaptive Traffic Signal Management
Vaibhav	PES1UG24CS640	Road Network and Vehicle Movement
Vainik	PES1UG24CS513	Congestion, Incident and Performance Management

Version 1.0

Prepared for the Software Engineering Mini-Project

1. Project Overview

The project is a C++ traffic-management simulation that models vehicles entering a road network, waiting in lane queues, moving through signal-controlled intersections, reacting to congestion or incidents, and producing measurable traffic-performance statistics. The goal is not to build a real city traffic controller; it is to build a clean, testable simulation that demonstrates software-engineering practice and appropriate use of data structures such as queues, graphs, priority queues and maps.

This plan is written for a four-member team starting from an empty GitHub repository. The work is divided so that every member owns one complete functional feature, contributes code that can be independently demonstrated, writes tests for the owned feature, and participates in integration and reviews. This avoids assigning a member only documentation, UI or testing work.

1.1 Project Objective

- Simulate traffic flow across a configurable road/intersection network.
- Use queue-based lane handling so waiting vehicles are processed in arrival order unless a valid priority rule applies.
- Control traffic signals using timed and queue-aware rules.
- Move vehicles from source to destination while respecting road capacity and signal states.
- Detect congestion and incidents and apply a simple management response.
- Collect useful statistics such as average waiting time, throughput, queue lengths and completed journeys.
- Build the system using modular C++ design, Git/GitHub collaboration, Agile planning, unit/integration/system tests, CI/CD and static analysis.

1.2 Guideline Alignment

The plan deliberately follows the course expectations: Agile development, active GitHub/Jira use, modular design, code reviews, CI/CD, security/validation planning, and visible implementation by every member. The project also prepares the required requirements, architecture/design, testing, validation and final-report outputs.

1.3 Scope

In Scope	Out of Scope
Console-driven traffic simulation with configuration files or menu input	Real GPS feeds, CCTV processing or live traffic APIs
Roads, lanes, intersections and traffic lights	A full graphical 3D city simulator
Vehicle generation and queueing	Microscopic physics such as acceleration curves or collision dynamics
Basic routing and movement across a graph	Machine-learning traffic prediction
Congestion/incident handling and emergency priority rules	Real-world legal deployment or hardware signal control
Statistics, reports and repeatable scenarios	Cloud-scale distributed simulation

1.4 Primary User

The primary user is a student/demo operator who configures a traffic scenario, starts the simulation, observes traffic-state summaries, injects or enables an incident scenario, and reviews the final performance report. The evaluator can also run deterministic predefined scenarios for repeatable testing.

2. Proposed Functional Requirements

ID	Requirement	Priority	Owner
FR-01	Create or load a simulation scenario containing intersections, roads, lane capacities and run duration.	High	Aarya / integration
FR-02	Generate vehicles using configurable arrival rates or a deterministic test schedule.	High	Aarya
FR-03	Assign each generated vehicle a unique ID, source, destination, class and route information.	High	Aarya
FR-04	Place vehicles into valid entry-lane queues without exceeding lane capacity.	High	Aarya
FR-05	Maintain signal phases for every controlled intersection.	High	Vijeth
FR-06	Advance signal timers and switch phases safely.	High	Vijeth
FR-07	Use queue information to extend/shorten a green phase within configured limits.	Medium	Vijeth
FR-08	Provide a controlled emergency-priority request that never enables conflicting greens.	Medium	Vijeth
FR-09	Represent the road network as connected nodes/edges with capacities and travel times.	High	Vaibhav
FR-10	Move eligible vehicles through green intersections and along their planned route.	High	Vaibhav
FR-11	Prevent movement into a full downstream road/lane.	High	Vaibhav
FR-12	Select a route using a simple shortest-path or weighted-path policy.	Medium	Vaibhav
FR-13	Measure lane/intersection congestion from queue length or occupancy thresholds.	High	Vainik
FR-14	Inject a road blockage/incident and mark affected road segments unavailable or restricted.	Medium	Vainik
FR-15	Trigger a rerouting or traffic-management response when an incident/congestion condition is detected.	Medium	Vainik
FR-16	Collect simulation metrics including throughput, waiting time, queue statistics and completed/unfinished trips.	High	Vainik
FR-17	Print or export a final text/CSV summary for the completed scenario.	Medium	Vainik

ID	Requirement	Priority	Owner
FR-18	Support deterministic seeds/configurations so tests and demos are reproducible.	High	Shared

2.1 Non-Functional Requirements

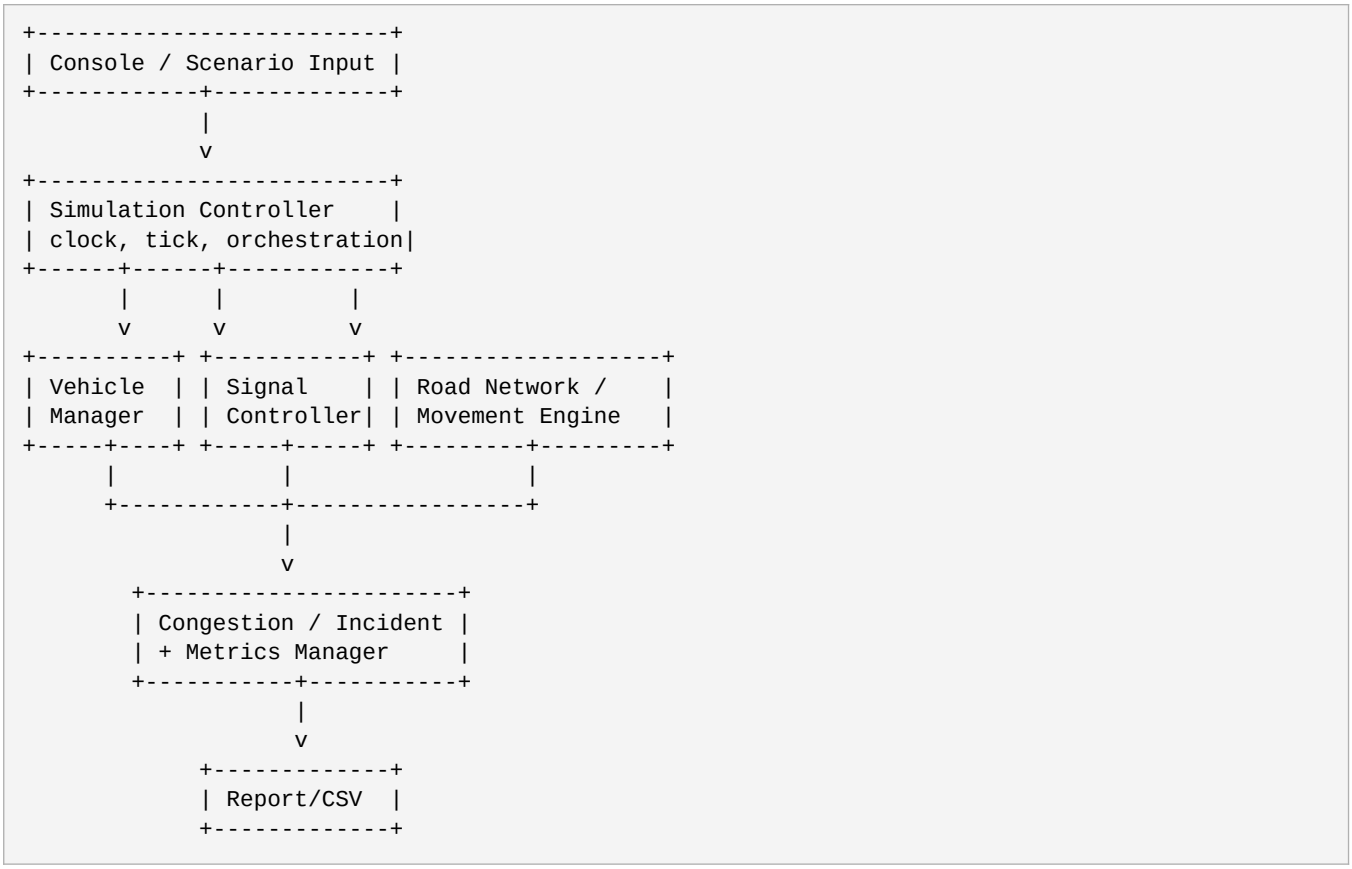
ID	Attribute	Requirement
NFR-01	Maintainability	Core modules must communicate through clear interfaces; avoid global mutable state except a controlled simulation context.
NFR-02	Testability	Each module must expose deterministic methods that can be tested without running the entire simulation.
NFR-03	Performance	A standard demo scenario of at least 4 intersections and 500 generated vehicles should complete comfortably on a normal student laptop without noticeable hangs.
NFR-04	Reliability	Invalid configuration must be rejected with clear error messages rather than causing undefined behavior.
NFR-05	Portability	Build using C++17 and CMake on Linux; avoid compiler-specific extensions.
NFR-06	Usability	Console commands/menu and report labels should be readable and consistent.
NFR-07	Security / robustness	Validate all file/menu inputs, reject impossible capacities/times, avoid unsafe C-style string handling and use RAII.
NFR-08	Traceability	Every implemented FR must map to source files and test IDs in the traceability matrix.
NFR-09	Code quality	Warnings enabled, formatting consistent, static analysis clean for critical findings.
NFR-10	Reproducibility	Random traffic generation must accept a seed and print/store the seed used.

2.2 Assumptions and Constraints

- The first version uses a discrete-time simulation. One simulation tick may represent one second or another fixed unit.
- A vehicle occupies one logical queue position; detailed physical dimensions are not simulated.
- Road capacities and travel times are simplified integer values.
- The network will be small enough for straightforward graph algorithms such as Dijkstra to be sufficient.
- The project is developed in C++17 with CMake and is expected to run primarily on Linux.
- The first deliverable should favor correctness, modularity and testability over visual complexity.

3. High-Level Architecture

Use a modular simulation architecture. The main program only handles startup, configuration and top-level commands. Business logic belongs in domain modules so that each feature can be developed and tested independently.



3.1 Core Data Structures

Concern	Recommended Structure	Reason
Lane waiting vehicles	std::queue<VehicleId> or std::deque<VehicleId>	Natural FIFO traffic queue; deque allows carefully controlled front insertion for priority policies if needed.
Road network	Adjacency list using vector/unordered_map	Efficient representation of a sparse road graph.
Routing frontier	std::priority_queue	Supports Dijkstra shortest-path computation.
Vehicle registry	std::unordered_map<VehicleId, Vehicle>	Fast lookup of vehicle state by ID.
Intersection registry	std::unordered_map<IntersectionId, Intersection>	Fast access to signal and lane state.
Metrics	Counters + vectors/running totals	Avoid storing unnecessary per-tick history unless needed for a report.
Events / incidents	vector or priority_queue by start tick	Allows scheduled events to trigger at specific simulation times.

3.2 Suggested C++ Classes

Class / Module	Key Responsibilities
Vehicle	id, source, destination, currentRoad, route, state, waitingTime, priorityClass
Road	id, from, to, capacity, travelTime, occupancy, enabled

Class / Module	Key Responsibilities
LaneQueue	capacity, queue; enqueue(), front(), pop(), size(), hasSpace()
Intersection	id, incoming/outgoing road IDs, lane queues, signal controller reference
SignalController	phase list, current phase, timer, min/max green; tick(), requestPriority()
RoadNetwork	addRoad(), neighbors(), findRoute(), isRoadAvailable()
VehicleManager	generateVehicle(), insertAtEntry(), validateVehicle(), scheduled arrivals
MovementEngine	advanceVehicles(), crossIntersection(), enterRoad(), completeTrip()
IncidentManager	scheduleIncident(), applyIncident(), clearIncident(), congestionStatus()
MetricsCollector	recordWait(), recordDeparture(), recordQueue(), buildSummary()
SimulationController	loadScenario(), step(), run(), reset(), stop condition

3.3 Tick Processing Order

1. Apply incidents/events scheduled for the current tick.
2. Generate vehicles whose arrival time is the current tick and place valid vehicles into entry queues.
3. Update congestion measurements from the current traffic state.
4. Update signal-controller timers and decide phase changes.
5. Move eligible vehicles in a deterministic order through intersections and along roads.
6. Update per-vehicle waiting/travel times and aggregate metrics.
7. Record optional snapshots and then advance the simulation clock.

Important integration rule: the processing order must be documented and stable. Otherwise two members may implement correct modules that produce inconsistent results when combined.

4. Equal Division of Work

Each member owns one complete functional slice from requirements to design, implementation, tests and demo. Shared project activities are rotated or divided evenly. No member is assigned only documentation, UI, testing or DevOps because the course expects implementation contribution from every team member.

Member	Owned Functionality	Requirements	Primary Code	Definition of Functional Completion	Owned Evidence
Aarya	Vehicle and Traffic Input Management	FR-01 to FR-04	Vehicle, LaneQueue, VehicleManager, input validation	Generate vehicles, assign attributes/routes, place them into valid lane queues, enforce capacity, support deterministic arrivals.	Unit tests for generation/capacity/invalid input; scenario configuration section; demo of traffic entering network.
Vijeth	Adaptive Traffic Signal Management	FR-05 to FR-08	SignalPhase, SignalController, signal policy	Timed phase switching, queue-aware extension, safe emergency priority, no conflicting greens.	Unit tests for phase sequence/timing/safety; signal design section; demo signal adaptation.
Vaibhav	Road Network and Vehicle Movement	FR-09 to FR-12	Road, RoadNetwork, MovementEngine, routing	Graph model, path selection, movement rules, downstream-capacity checks, trip completion.	Routing and movement tests; graph/algorithm design; demo vehicle progression across multiple intersections.

Member	Owned Functionality	Requirements	Primary Code	Definition of Functional Completion	Owned Evidence
Vainik	Congestion, Incident and Performance Management	FR-13 to FR-17	IncidentManager, MetricsCollector, report writer	Detect congestion, inject/clear incidents, request management response, collect metrics, final report/CSV.	Incident/metric tests; validation/report section; demo blockage + before/after metrics.

4.1 Shared Responsibilities - Still Equal

Activity	Division Rule
Requirements review	All four review all FR/NFRs; each member writes/refines requirements for the owned module.
Architecture	All four agree interfaces before implementation; Vaibhav documents graph interfaces, Vijeth signal interface, Aarya vehicle/input interface, Vainik metrics/incident interface.
Code reviews	Every PR requires one teammate reviewer. Review pairs rotate so the same person is not always reviewing the same teammate.
Integration	Each member integrates the owned module and fixes integration defects caused by that module; cross-module defects are paired.
Testing	Each member writes unit tests for owned functionality; integration/system tests are split across the four members.
Jira/GitHub	Each member maintains their own stories/tasks and links PRs/commits. One person may create the board, but ownership remains individual.
Presentation	Each member presents approximately one quarter: problem/requirements, architecture + module, code/tests, and demo evidence for owned functionality.

4.2 Review Pair Rotation

Sprint	Review Rotation
Sprint 1	Aarya reviews Vijeth; Vijeth reviews Vaibhav; Vaibhav reviews Vainik; Vainik reviews Aarya
Sprint 2	Aarya reviews Vaibhav; Vaibhav reviews Vijeth; Vijeth reviews Vainik; Vainik reviews Aarya
Sprint 3	Aarya reviews Vainik; Vainik reviews Vaibhav; Vaibhav reviews Vijeth; Vijeth reviews Aarya

5. GitHub Repository Plan - Starting From Empty

5.1 Initial Repository Structure

```
traffic-management-system/
|-- CMakeLists.txt
|-- README.md
|-- LICENSE                          # optional if course permits
|-- .gitignore
|-- .clang-format
```

```
|-- Dockerfile
|-- Jenkinsfile
|-- config/
|   |-- demo_scenario.txt
|   `-- test_scenario.txt
|-- include/
|   |-- model/
|   |-- traffic/
|   |-- signal/
|   |-- network/
|   |-- incident/
|   |-- metrics/
|   `-- simulation/
|-- src/
|   |-- main.cpp
|   |-- model/
|   |-- traffic/
|   |-- signal/
|   |-- network/
|   |-- incident/
|   |-- metrics/
|   `-- simulation/
|-- tests/
|   |-- unit/
|   |-- integration/
|   `-- system/
|-- docs/
|   |-- requirements/
|   |-- design/
|   |-- test-plan/
|   `-- validation/
`-- scripts/
    |-- run_demo.sh
    `-- coverage.sh
```

5.2 First Commit Sequence

Do not let one person build the whole skeleton and leave everyone else with invisible contribution. Create the minimum skeleton in one setup PR, then let each member add the initial interface for the owned module in a separate PR.

```
# Replace the URL with the actual repository URL.
git clone <repository-url>
cd <repository-name>

git checkout -b setup/project-skeleton
mkdir -p include src tests/unit tests/integration tests/system config docs scripts
touch README.md .gitignore CMakeLists.txt src/main.cpp Dockerfile Jenkinsfile

git add .
git commit -m "chore: initialize C++ project structure"
git push -u origin setup/project-skeleton
# Open PR -> review -> merge into main.
```

After the skeleton merge, every member creates a feature branch and contributes a compilable interface + starter test for the owned module. This gives all four members meaningful early commits.

5.3 Branch Naming

Pattern / Example	Purpose
setup/...	Project skeleton, build configuration, CI bootstrap
feature/aarya-vehicle-input	Aarya vehicle generation, lane queue and traffic input work
feature/vijeth-signal-control	Vijeth signal-controller work
feature/vaibhav-network-movement	Vaibhav graph, routing and movement work
feature/vainik-incident-metrics	Vainik congestion, incident and metrics work
test/...	Cross-module test additions
fix/...	Bug fixes after integration
docs/...	Documentation-only updates

5.4 Commit Message Convention

- feat: add deterministic vehicle generator
- feat: implement signal phase timer
- fix: prevent movement into full downstream lane
- test: cover incident activation and clearing
- docs: add routing design notes
- ci: run gtest and coverage in Jenkins
- refactor: extract lane capacity validation

Commits should represent real logical changes. Do not create meaningless commits only to increase contribution counts. The objective is visible, understandable ownership.

5.5 Pull Request Rules

- No direct feature work on main.
- PR title should describe the user-visible or technical change.
- PR description must mention requirement IDs implemented, tests added, and known limitations.
- At least one teammate reviews before merge.
- Build, unit tests and static analysis must pass before merge.
- Large PRs should be split; aim for reviewable changes rather than one final dump.
- Resolve review comments in the PR rather than through private messages so the process is visible.

6. Agile and Jira Plan

Use a simple Scrum-style workflow with short sprints. The purpose of Jira is traceability, not administrative overload. Every story should be small enough to finish and demonstrate within a sprint.

6.1 Suggested Board Columns

Backlog -> Selected for Sprint -> In Progress -> In Review -> Testing -> Done

6.2 Epics

Epic	Name	Owner
EPIC-1	Requirements, Planning and Architecture	Shared
EPIC-2	Vehicle and Traffic Input Management	Aarya
EPIC-3	Adaptive Traffic Signal Management	Vijeth

Epic	Name	Owner
EPIC-4	Road Network and Vehicle Movement	Vaibhav
EPIC-5	Congestion, Incident and Performance Management	Vainik
EPIC-6	Integration, Validation and Delivery	Shared

6.3 Initial Backlog - Create These Issues

Key	Issue	Type	Owner	Epic
SE-01	Agree simulation assumptions and tick order	Task	All	EPIC-1
SE-02	Write initial SRS requirements and IDs	Story	All	EPIC-1
SE-03	Create CMake project skeleton	Task	Rotate/setup	EPIC-1
SE-04	Define shared IDs and domain types	Task	All	EPIC-1
SE-05	Implement Vehicle model and validation	Story	Aarya	EPIC-2
SE-06	Implement lane queue with capacity rules	Story	Aarya	EPIC-2
SE-07	Implement deterministic vehicle generator	Story	Aarya	EPIC-2
SE-08	Implement signal phase model	Story	Vijeth	EPIC-3
SE-09	Implement fixed-time signal transitions	Story	Vijeth	EPIC-3
SE-10	Add queue-aware signal adaptation	Story	Vijeth	EPIC-3
SE-11	Implement road/intersection graph	Story	Vaibhav	EPIC-4
SE-12	Implement route calculation	Story	Vaibhav	EPIC-4
SE-13	Implement movement across roads/intersections	Story	Vaibhav	EPIC-4
SE-14	Implement congestion detector	Story	Vainik	EPIC-5
SE-15	Implement incident activation and clearing	Story	Vainik	EPIC-5
SE-16	Implement metrics collector and summary	Story	Vainik	EPIC-5
SE-17	Integrate vehicle -> signal -> movement tick	Integration	All	EPIC-6
SE-18	Add integration tests for standard scenario	Test	Split	EPIC-6
SE-19	Add Jenkins pipeline + Docker build	DevOps	Split	EPIC-6
SE-20	Add static analysis and coverage report	DevOps/Test	Split	EPIC-6
SE-21	Run system validation scenarios	Validation	All	EPIC-6

Key	Issue	Type	Owner	Epic
SE-22	Complete final report, demo and presentation	Documentation	All	EPIC-6

7. Sprint-by-Sprint Roadmap

The schedule below is relative because an exact submission date was not provided. If the course has fewer weeks, keep the same order and compress the sprints; do not skip the requirements/design/test evidence.

Period	Goal	Team Output	Equal Contribution Rule
Sprint 0 - Week 1	Foundation	Finalize SRS draft, module interfaces, repo skeleton, board, coding standard, first build.	Each member contributes requirements + header/interface + starter unit test for owned module.
Sprint 1 - Weeks 2-3	Minimum working simulation	Generate vehicles, fixed signals, graph + movement, basic metrics; run one deterministic end-to-end scenario.	All four functional modules present in minimum form.
Sprint 2 - Weeks 4-5	Full functionality	Queue-aware signals, shortest path, incidents, congestion response, stronger input validation and final report output.	Each owner completes the full promised functionality.
Sprint 3 - Week 6	Integration and test hardening	Integration/system tests, defect fixes, coverage, invalid-input cases, reproducibility.	Pair debugging and independent PR reviews.
Sprint 4 - Week 7	CI/CD and validation	Jenkins, Docker, static analysis, validation report, traceability matrix, maintenance plan.	DevOps tasks divided; every member supplies evidence for own module.
Week 8	Final demo and presentation	Stable tagged release, demo scenarios, final report, slides, 15-minute rehearsal.	Each member presents owned functionality and code/test evidence.

7.1 Per-Member Sprint Tasks

Member	Detailed Sprint Ownership
Aarya	S0: Vehicle types + queue interface. S1: generator + capacity. S2: configurable arrivals + validation. S3: edge tests + integration fixes. S4: coverage/static-analysis fixes + SRS traceability. Final: input/queue demo.
Vijeth	S0: Signal states/interfaces. S1: fixed-time phases. S2: queue-aware adaptation + priority safety. S3: timing/integration tests. S4: CI failures/static-analysis fixes + design traceability. Final: signal demo.
Vaibhav	S0: Graph/road interfaces. S1: road network + basic movement. S2: route calculation + capacity-aware movement. S3: multi-intersection integration tests. S4: Docker/build reproducibility + design traceability. Final: routing/movement demo.
Vainik	S0: Metrics/incident interfaces. S1: basic metrics. S2: congestion + incidents + CSV summary. S3: validation scenarios + defect fixes. S4: Jenkins/report artifacts + validation traceability. Final: incident/metrics demo.

8. Work Breakdown Structure and Effort Estimate

Effort estimates are rough planning values. For consistency, this document uses 1 person-month (PM) = approximately 160 hours. Actual student project time will be much lower than a full industrial month, so the PM values are fractional and are used only to satisfy planning/estimation practice.

WBS	Work Package	Owner	Hours	PM
1.0	Requirements and SRS	All	20	0.125
1.1	Functional/NFR definition and traceability IDs	All	8	0.050
1.2	Use cases and analysis model	All	8	0.050
1.3	Validation planning	All	4	0.025
2.0	Architecture and design	All	24	0.150
2.1	Shared domain interfaces + tick order	All	8	0.050
2.2	Per-module class/design details	Each owner	12	0.075
2.3	Design review and fixes	All	4	0.025
3.0	Aarya feature implementation	Aarya	36	0.225
4.0	Vijeth feature implementation	Vijeth	36	0.225
5.0	Vaibhav feature implementation	Vaibhav	36	0.225
6.0	Vainik feature implementation	Vainik	36	0.225
7.0	Integration	All	24	0.150
8.0	Testing and coverage	All	32	0.200
9.0	CI/CD, Docker, static analysis	All split	16	0.100
10.0	Validation + maintenance plan	All	16	0.100
11.0	Final report and presentation	All	20	0.125

Equal implementation check: each primary functional owner is allocated the same implementation estimate (36 hours / 0.225 PM), with shared activities divided among all four members.

8.1 Simple Gantt Schedule

Activity	W1	W2	W3	W4	W5	W6	W7	W8
Requirements / SRS	X							
Architecture / interfaces	X	X						
Aarya feature		X	X	X	X			

Activity	W1	W2	W3	W4	W5	W6	W7	W8
Vijeth feature		X	X	X	X			
Vaibhav feature		X	X	X	X			
Vainik feature		X	X	X	X			
Integration			X	X	X	X		
Unit/ integration/ system testing		X	X	X	X	X	X	
CI/CD + static analysis			X	X	X	X	X	
Validation report / maintenance						X	X	
Final report + demo							X	X

9. Testing Strategy

The course test plan expects unit, integration and system testing, including valid and invalid inputs. The final test-plan document should use unique IDs and record actual result plus pass/fail status after execution. Build the tests continuously rather than writing them after implementation.

9.1 Unit-Test Ownership

Owner	Modules	Planned IDs
Aarya	VehicleManager / LaneQueue	UT-A01 to UT-A10
Vijeth	SignalController	UT-VI01 to UT-VI10
Vaibhav	RoadNetwork / MovementEngine	UT-VA01 to UT-VA10
Vainik	IncidentManager / MetricsCollector	UT-VN01 to UT-VN10

9.2 Planned Test Scenarios - Aarya

- UT-A01: valid vehicle is generated with a unique ID.
- UT-A02: deterministic seed produces the same arrival sequence.
- UT-A03: invalid source or destination is rejected.
- UT-A04: vehicle enters a lane when capacity is available.
- UT-A05: vehicle is rejected/deferred when lane is full.
- UT-A06: FIFO ordering is preserved for normal vehicles.
- UT-A07: zero/negative configuration values are rejected.
- UT-A08: scheduled arrival occurs at the correct simulation tick.
- UT-A09: generated route request contains correct source/destination.
- UT-A10: reset clears generator and queue state correctly.

9.3 Planned Test Scenarios - Vijeth

- UT-VI01: signal starts in the configured initial phase.
- UT-VI02: signal changes after the configured green duration.
- UT-VI03: yellow/all-red transition is respected if modeled.
- UT-VI04: conflicting approaches are never green simultaneously.

- UT-VI05: queue-aware extension stays below maximum green.
- UT-VI06: low-demand approach does not starve beyond configured limit.
- UT-VI07: emergency priority request is accepted only when safe.
- UT-VI08: invalid timing configuration is rejected.
- UT-VI09: reset restores initial phase and timer.
- UT-VI10: identical queue inputs produce deterministic phase decisions.

9.4 Planned Test Scenarios - Vaibhav

- UT-VA01: road and intersection are inserted into graph correctly.
- UT-VA02: shortest route is returned on a simple known graph.
- UT-VA03: disabled road is not used for routing.
- UT-VA04: unreachable destination returns a controlled failure.
- UT-VA05: vehicle moves only when the signal permits.
- UT-VA06: vehicle does not enter a full downstream road/lane.
- UT-VA07: vehicle reaches destination and is marked complete.
- UT-VA08: travel-time delay is respected before next movement.
- UT-VA09: multiple vehicles preserve queue/movement consistency.
- UT-VA10: reset clears occupancy and in-transit state.

9.5 Planned Test Scenarios - Vainik

- UT-VN01: congestion threshold triggers at the expected queue level.
- UT-VN02: congestion status clears when traffic falls below recovery threshold.
- UT-VN03: incident activates exactly at configured tick.
- UT-VN04: incident marks the correct road restricted/unavailable.
- UT-VN05: incident clears at configured end tick.
- UT-VN06: reroute/management request is emitted after incident detection.
- UT-VN07: throughput increments only for completed journeys.
- UT-VN08: average waiting time is computed correctly from a known sample.
- UT-VN09: queue maximum and average statistics are recorded correctly.
- UT-VN10: report/CSV contains expected fields and no divide-by-zero errors.

9.6 Integration and System Tests

ID	Scenario	Owners
IT-01	Vehicle generator -> lane queue -> signal -> movement path works end to end.	Aarya + Vijeth
IT-02	Signal green causes eligible queued vehicles to move without exceeding downstream capacity.	Vijeth + Vaibhav
IT-03	Road incident changes routing/movement behavior.	Vaibhav + Vainik
IT-04	Metrics accurately reflect completed movement after a deterministic 100-tick run.	Aarya + Vainik
ST-01	Normal balanced traffic scenario completes with no errors.	All
ST-02	Rush-hour scenario creates congestion and queue-aware behavior.	All
ST-03	Blocked-road scenario triggers incident response and route change.	All
ST-04	Emergency-priority scenario changes signal safely and allows priority progress.	All

ID	Scenario	Owners
ST-05	Invalid configuration is rejected and program exits cleanly.	All

9.7 Coverage Targets

- Aim for at least 80% line coverage in core domain modules if practical; report the actual measured value rather than claiming a number that was not measured.
- Prioritize branch coverage for safety-critical decisions: signal conflicts, full lanes, invalid input, unreachable routes and incident states.
- Coverage is evidence, not a substitute for good assertions. Every test should verify behavior.

10. CI/CD, Docker and Static Analysis

10.1 Jenkins Pipeline Stages

Stage	Action
1. Checkout	Pull the PR/branch source.
2. Configure	cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
3. Build	cmake --build build --parallel
4. Unit tests	ctest --test-dir build --output-on-failure
5. Static analysis	Run cppcheck and/or SonarQube scanner as configured.
6. Coverage	Run gcov/lcov or llvm-cov and archive the report.
7. Docker build	Build the reproducible runtime image.
8. System smoke test	Run one deterministic scenario inside the container.
9. Archive	Store test and coverage artifacts.

10.2 DevOps Task Split

Member	Owned CI/CD Contribution
Aarya	CMake test discovery + coverage script
Vijeth	Static-analysis configuration and warning cleanup
Vaibhav	Dockerfile and reproducible container run command
Vainik	Jenkinsfile pipeline wiring and artifact archiving

10.3 Recommended Build Quality Flags

```
-std=c++17
-Wall -Wextra -Wpedantic
# Optional during development if the codebase is warning-clean:
-Werror
```

Use sanitizers locally when possible (AddressSanitizer/UndefinedBehaviorSanitizer) for robustness. If the classroom environment cannot support a tool, document that constraint rather than silently omitting the check.

11. Coding and Integration Rules

- Prefer small classes with one responsibility.
- Use header/source separation for reusable modules.
- Use const correctness and pass large objects by const reference where appropriate.
- Use smart pointers only where ownership requires dynamic allocation; prefer values/references for simple domain objects.
- Do not expose internal containers directly. Provide methods such as enqueue(), size(), hasSpace(), neighbors().
- Avoid global variables. Keep simulation state in the controller/context.
- Centralize shared IDs and time types to avoid mismatched int/string conventions.
- All randomness goes through one seeded generator service or controlled generator object.
- Public methods that can fail must return a clear status/result or throw only documented exceptions.
- Every bug fix should add or strengthen a regression test whenever possible.

11.1 Interface Freeze Before Deep Implementation

Before Sprint 1 code grows, agree the minimum shared interfaces. For example, Aarya should not need to know the internal signal state machine, and Vijeth should not access Vaibhav's graph containers directly. Integration should occur through methods such as getQueueLength(intersection, approach), canEnterRoad(roadId), requestRoute(source, destination), and recordVehicleCompleted(vehicleId).

12. Requirements Traceability Starter Matrix

Requirement	Brief Description	Code Reference	Unit Tests	System Tests
FR-02	Vehicle generation	VehicleManager.cpp	UT-A01, UT-A02	ST-01
FR-04	Lane capacity / queueing	LaneQueue.cpp	UT-A04, UT-A05, UT-A06	ST-02
FR-06	Signal timing	SignalController.cpp	UT-VI02, UT-VI04	ST-01
FR-07	Queue-aware adaptation	SignalController.cpp	UT-VI05, UT-VI06	ST-02
FR-10	Vehicle movement	MovementEngine.cpp	UT-VA05, UT-VA06, UT-VA07	ST-01
FR-12	Route selection	RoadNetwork.cpp	UT-VA02, UT-VA03, UT-VA04	ST-03
FR-14	Incident management	IncidentManager.cpp	UT-VN03, UT-VN04, UT-VN05	ST-03
FR-16	Metrics	MetricsCollector.cpp	UT-VN07, UT-VN08, UT-VN09	ST-01, ST-02

Update this matrix as filenames and test IDs stabilize. The final report should contain the complete mapping for all implemented requirements.

13. Course Deliverables and Who Prepares What

Deliverable	Contents	Ownership
Synopsis / Proposal	Project title, team profile, project description, feature list, short-term plan, who-does-what and product ownership.	All; each owner writes own feature section.
Phase 1 - SRS	Purpose, scope, users, environment, interfaces, analysis models, functional/NFRs, glossary, field layouts if relevant, traceability matrix.	Sections split; final review by all.

Deliverable	Contents	Ownership
Phase 2 - Project Plan	Lifecycle justification, tool choices, reuse/build decisions, WBS, effort estimates, Gantt.	Use this document as the starting point; all approve.
Architecture Document	High-level components, module boundaries, interfaces, major data flow, deployment/build view.	All; each owner documents module interface.
Software Design Document	Class diagrams, sequence/activity diagrams for use cases, algorithm notes, APIs/ABIs where relevant.	All; feature-specific diagrams by owner.
Phase 3 - Test Plan	8-10 tests per implemented use case, with unit/integration/system entries, expected/actual/result.	Each owner supplies full cases for own feature; shared integration cases split.
Validation Report	System validation results, defects, security/robustness checks, traceability, limitations.	All, coordinated from system-test evidence.
Maintenance Plan	How to add roads, signal policies, metrics, new scenarios; known technical debt and upgrade path.	All.
Final Report	Proposal, SRS, plan, design diagrams, test cases, output screenshots, validation evidence.	All.
Demo + 15-minute presentation	Show use cases, code, tests, CI evidence and individual contribution.	Approximately 3-4 minutes per member.

13.1 Suggested Presentation Split

Member	Presentation Segment
Aarya	Problem statement, major requirements, input/vehicle flow demo, selected unit tests.
Vijeth	Architecture link to signal module, adaptive signal logic, safety checks, code walkthrough.
Vaibhav	Graph/routing/movement design, data structures, multi-intersection movement demo.
Vainik	Congestion/incident scenario, metrics results, CI/CD/validation summary and limitations.

14. Risks and Mitigation

ID	Risk	Impact	Mitigation
R1	Modules do not integrate because data types/interfaces differ	High	Freeze shared interfaces early; use one header for shared IDs/types; integration test in Sprint 1.
R2	One member finishes much later than others	Medium	Break feature into small Jira stories, review progress each sprint, pair on blockers without transferring ownership.
R3	Simulation becomes too complex	High	Keep discrete-time model and simple capacities; defer GUI/ML/live data.
R4	Random behavior makes tests flaky	High	Use deterministic seeds and predefined test schedules.

ID	Risk	Impact	Mitigation
R5	Signal logic creates impossible/conflicting greens	High	Explicit allowed phases + invariant tests.
R6	Routing and incident behavior loops indefinitely	Medium	Track unreachable routes, max hops/retry policies, and fail safely.
R7	CI tools fail on student machines	Medium	Keep local CMake/gtest path independent; containerize Jenkins build where practical; document tool constraints.
R8	Git history does not show equal contribution	High	Feature ownership, separate PRs, regular meaningful commits, cross-review evidence.
R9	Documentation is written at the end	High	Update docs in the same sprint as code; link requirement IDs in PRs.

15. Definition of Done

A Jira story can move to Done only when all applicable points below are satisfied:

- Requirement or acceptance criteria are clear and linked.
- Implementation builds with the agreed compiler settings.
- Relevant unit tests are added and pass.
- No new critical static-analysis findings are introduced.
- Public interfaces are documented enough for teammates to use them.
- PR has at least one teammate review and all requested changes are resolved.
- Feature is integrated with main without breaking existing tests.
- Traceability row/test IDs are updated when the change completes a requirement.
- The owner can explain and demonstrate the code without relying on generated notes.

16. Recommended Demo Scenarios

Scenario	Setup	What to Demonstrate
Demo 1 - Normal Flow	4-intersection network, balanced arrivals, fixed/queue-aware signals.	Show queues, movement and baseline throughput.
Demo 2 - Rush Hour	High arrival rate on one corridor.	Show congestion detection and signal adaptation; compare wait/queue metrics.
Demo 3 - Road Blockage	Block one road at a known tick.	Show incident activation and changed route/traffic response.
Demo 4 - Emergency Priority	Introduce one priority vehicle or request.	Show safe signal priority without conflicting greens.

17. What the Team Should Do Now - Empty Repository

Do the following in order. This is the practical starting checklist for the team.

1. Create/confirm the GitHub repository and add all four members with collaborator access.
2. Protect main if possible: require pull requests and at least one review.
3. Create the Jira board with the six epics and the initial backlog listed in this document.
4. Agree on the exact simulation assumptions: tick duration, road capacity meaning, whether yellow/all-red is modeled, and how emergency priority works.

5. Create a setup/project-skeleton branch and add CMakeLists.txt, README.md, .gitignore, src/main.cpp, include/, tests/, config/, docs/, Dockerfile and Jenkinsfile placeholders.
6. Merge the skeleton only after one teammate reviews it.
7. Each member creates the named feature branch and adds a header/interface plus one starter unit test for the owned module.
8. Hold a 30-minute interface review. Confirm shared ID types and method signatures before deep coding.
9. Implement the minimum Sprint 1 path: one vehicle generated -> queued -> green signal -> movement -> destination -> metric increment.
10. Only after the minimum end-to-end path works, add adaptation, incidents, rerouting and richer reports.
11. At the end of every sprint, merge only passing PRs, tag/record a demo state, update SRS/design/test evidence and move Jira issues to Done.

17.1 Day-One Task Split

Who	Day-One Output
Aarya	Draft Vehicle/LaneQueue interfaces; add FR-01 to FR-04 acceptance criteria; create vehicle starter tests.
Vijeth	Draft SignalController/phase interfaces; define safe phase table; add FR-05 to FR-08 acceptance criteria.
Vaibhav	Draft Road/RoadNetwork/Movement interfaces; define graph representation; add FR-09 to FR-12 acceptance criteria.
Vainik	Draft Incident/Metrics interfaces; define metrics formulas/incident schedule; add FR-13 to FR-17 acceptance criteria.
All	Review shared types and tick order; agree naming; approve repo/branch/PR rules.

17.2 Minimum README Contents

- Project title and one-paragraph objective.
- Team members and feature ownership.
- Prerequisites: C++17 compiler, CMake, testing tool.
- Build commands.
- Run command with sample configuration.
- Test command.
- Repository structure.
- Branch/PR convention.
- Current implemented features and known limitations.

18. Lifecycle and Tool Justification

18.1 Lifecycle

Use Agile iterative development. The system has several interacting modules whose interfaces will become clearer after the first working simulation. Short sprints allow the team to implement a small end-to-end path early, receive feedback, then add adaptive signal control, routing and incident handling without postponing integration until the end.

18.2 Tools

Purpose	Tool	Reason
Planning / tracking	Jira (or the course-approved equivalent)	Backlog, stories, sprint status, ownership and defect tracking.

Purpose	Tool	Reason
Version control	Git + GitHub	Individual commits, branches, PRs, reviews and history.
Language	C++17	Matches assigned project; suitable for queues, graphs and efficient simulation.
Build	CMake	Portable multi-file build and test integration.
Unit testing	GoogleTest	Standard C++ assertions, fixtures and CMake integration.
CI/CD	Jenkins	Automates build, tests, analysis, coverage and demo packaging.
Containerization	Docker	Reproducible build/runtime for CI and demo.
Static analysis	SonarQube and/or cppcheck	Detects maintainability and common C++ defects.
Coverage	gcov/lcov or llvm-cov	Evidence for unit-test coverage.
Design diagrams	draw.io / PlantUML / similar	Use case, class, sequence and component diagrams.

18.3 Build vs Reuse Components

Category	Component	Justification
Build	Vehicle manager, lane queue, signal controller, road network, movement engine, incident manager, metrics logic, simulation controller	These are the educational core and must demonstrate team implementation.
Reuse	C++ Standard Library containers/algorithms	Standard library use is expected; reimplementing queue/map/priority_queue would add risk without project value.
Reuse	GoogleTest	Testing framework, not project business logic.
Reuse	CMake, Jenkins, Docker, SonarQube/cppcheck	Engineering infrastructure required to build, test and validate the project.

19. Final Submission Checklist

- Synopsis contains the project title, team names/SRNs, description, functional features, work plan and clear product ownership.
- SRS contains measurable functional/NFR requirements and analysis models.
- Project plan contains lifecycle justification, tool choices, build/reuse classification, WBS, effort estimate and Gantt.
- Architecture and design documents show module boundaries and interfaces.
- Each of the four members has meaningful code commits and at least one complete functional feature.
- Pull requests and code reviews are visible in GitHub.
- Jira/backlog shows stories, owners, sprint progress and defects.
- Unit, integration and system tests are documented and executed; actual results/pass-fail are recorded.
- Coverage and static-analysis evidence are captured.
- Jenkins pipeline and Docker build/run are demonstrated or documented with evidence.
- Validation report, maintenance plan and requirement traceability matrix are complete.
- Final report contains screenshots/output from deterministic demo scenarios.
- All four members can explain the architecture, own code, tests and how their module integrates with the rest of the system.

20. One-Page Ownership Summary

Member	Functional Slice	Core Output	Requirement IDs	Unit Tests	Demo Proof
Aarya	Vehicle and traffic input	Vehicle generation, valid inputs, lane queues, capacity	FR-01 to FR-04	UT-A01..A10	Traffic enters simulation correctly
Vijeth	Traffic signals	Phase timing, adaptive green, priority safety	FR-05 to FR-08	UT-VI01..VI10	Queues are released safely by signal logic
Vaibhav	Road network and movement	Graph, route finding, road capacity, movement	FR-09 to FR-12	UT-VA01..VA10	Vehicles traverse multiple intersections
Vainik	Congestion, incidents, metrics	Thresholds, blockages, management response, stats/report	FR-13 to FR-17	UT-VN01..VN10	System reacts to disruption and reports performance

This division is intentionally balanced: four implementation-heavy functional slices, equal estimated implementation effort, equal test ownership, shared integration duties and approximately equal final-presentation time.

21. Notes to Fill Before Submission

- Actual GitHub repository URL.
- Actual Jira project key/board link.
- Exact course deadlines and dates in the Gantt chart.
- Final architecture/design diagrams after interfaces are approved.
- Measured coverage percentages and validation results after implementation.